

Introdução REACT - MATERIAL AUTOINSTRUCIONAL - SEMANA 01

Introdução ao React e Configuração do Ambiente

Unidade Curricular: Desenvolvimento de Frameworks - React

Projeto: To-Do Pro - Gestão de tarefas fullstack

Carga sugerida: 3h45min

Nível: Introdução

1. Como estudar este material

Este material foi preparado para que você consiga avançar mesmo quando estiver estudando sem acompanhamento constante. Leia a explicação, execute os comandos, faça os exercícios na ordem e só avance depois de verificar o resultado.

Durante a prática:

1. Crie uma pasta exclusiva para seus projetos.
2. Digite os comandos em vez de apenas copiá-los.
3. Leia as mensagens do terminal e do navegador.
4. Salve pequenas mudanças com frequência.
5. Registre dúvidas e erros no seu caderno ou em `ERROS.md`.

Ao terminar, você deverá ter uma aplicação React executável, um repositório Git e uma primeira tela do To-Do Pro.

2. O desafio do To-Do Pro

Uma pequena empresa organiza suas tarefas em planilhas e mensagens. Esse processo causa tarefas duplicadas, informações desatualizadas, dificuldade para saber os responsáveis e ausência de histórico.

Você participará da equipe que desenvolverá o **To-Do Pro**, uma aplicação para cadastrar, acompanhar e concluir tarefas. Nesta semana, o objetivo é preparar a base técnica e entregar a primeira tela funcional.

3. Objetivos de aprendizagem

Ao finalizar o material, você deverá conseguir:

- explicar frontend, backend e fullstack;
- explicar por que o React é usado no frontend;
- verificar Node.js, npm e Git pelo terminal;
- criar um projeto React com Vite;
- iniciar e encerrar um servidor local;
- identificar arquivos importantes do projeto;
- escrever um componente funcional usando JSX;
- registrar seu trabalho com Git;
- documentar como outra pessoa pode executar o projeto;
- investigar problemas simples de configuração.

4. Conceitos fundamentais

4.1 Frontend, backend e fullstack

O **frontend** é a parte da aplicação com a qual o usuário interage: páginas, textos, botões, formulários e mensagens.

O **backend** executa regras de negócio no servidor. Ele pode validar dados, controlar permissões, processar requisições e conversar com o banco de dados.

O **banco de dados** armazena informações de forma persistente. No To-Do Pro, ele será usado futuramente para guardar tarefas mesmo depois que o navegador for fechado.

Uma aplicação **fullstack** conecta essas partes:

None

Usuário -> Interface React -> API do backend -> Banco de dados

Usuário <- Interface React <- Resposta da API <- Banco de dados

4.2 O que é React?

React é uma biblioteca JavaScript para criação de interfaces de usuário. Em vez de escrever a página inteira como um único bloco, dividimos a interface em componentes menores, como cabeçalho, formulário, item de tarefa e lista.

O React trabalha com uma abordagem declarativa: você descreve como a interface deve aparecer para determinado estado, e a biblioteca atualiza a tela quando os dados mudam.

4.3 SPA e componentes

Uma **SPA**, ou Single Page Application, carrega uma aplicação inicial e atualiza partes da interface sem recarregar todo o documento a cada interação.

A estrutura futura do To-Do Pro poderá ser:

```
None
App
├── Header
├── TaskForm
├── TaskList
    └── TaskItem
```

4.4 React e Angular: pontos de contato e diferenças

Como você já conhece Angular, use esse conhecimento como uma ponte, mas evite assumir que React funciona com as mesmas abstrações. Os dois ecossistemas constroem interfaces reativas e baseadas em componentes, porém fazem escolhas diferentes sobre estrutura, atualização e gerenciamento de estado.

Conceito	Angular	React
Natureza	Framework completo, com convenções e recursos integrados.	Biblioteca de UI; roteamento, requisições e estado global são escolhidos separadamente.
Componente	Normalmente combina classe TypeScript, template, estilos e metadados.	Geralmente é uma função JavaScript/TypeScript que retorna JSX.
Template	Usa HTML com diretivas como <code>*ngIf</code> , <code>*ngFor</code> e bindings.	Usa JSX, misturando marcação e expressões JavaScript.

Binding	Possui property binding, event binding e two-way binding com <code>[(ngModel)]</code> .	O fluxo padrão é unidirecional; campos controlados usam <code>value</code> e <code>onChange</code> .
Estado local	Pode estar em propriedades da classe, Signals ou outras soluções.	É comum usar hooks, como <code>useState</code> e <code>useReducer</code> .
Injeção de dependência	Faz parte do framework e é usada por serviços.	Não existe um contêiner DI obrigatório; dependências são passadas por props, módulos ou bibliotecas.
Diretivas	Extendem o comportamento ou a renderização do template.	O comportamento costuma ser expresso por componentes, hooks e JavaScript.
Atualização	Angular executa seu mecanismo de detecção de mudanças/signals.	React reexecuta componentes e reconcilia a árvore de elementos resultante.

No Angular, é comum pensar primeiro em um componente com template, classe e ciclo de vida. No React, pense em uma função que recebe entradas, lê estado e devolve uma descrição da interface:

None

```
UI = função(props, state)
```

Essa expressão não significa que o DOM seja recriado de forma ingênua a cada clique. Significa que o código do componente descreve o resultado esperado. O React compara a nova descrição com a anterior e aplica ao DOM real somente as alterações necessárias.

Atenção: JSX não é um template HTML que o navegador interpreta diretamente. O Vite transforma JSX em JavaScript durante o desenvolvimento e o build. Por isso, expressões JavaScript aparecem entre chaves e regras de JavaScript, como nomes de variáveis e funções, fazem parte da escrita da interface.

4.5 O DOM real e a árvore de elementos

O **DOM**, Document Object Model, é a representação em árvore que o navegador cria a partir do documento HTML. Cada elemento vira um nó relacionado a outros nós:

None

```
<main>
  <h1>To-Do Pro</h1>
  <button>Nova tarefa</button>
</main>
```

Pode ser representado assim:

```
None
document
└─ main
    ├── h1
    │   └─ "To-Do Pro"
    └─ button
        └─ "Nova tarefa"
```

O navegador mantém esse DOM em memória. Alterar propriedades, textos e classes pode provocar trabalho de layout e pintura visual. Uma alteração isolada é barata, mas muitas alterações imperativas espalhadas pelo código tornam o fluxo difícil de prever.

No JavaScript tradicional, poderíamos escrever:

```
None
const title = document.querySelector("h1");
title.textContent = "Tarefas de hoje";
title.classList.add("highlight");
```

Esse código procura e modifica diretamente o DOM. Ele é válido, mas a aplicação precisa controlar manualmente qual elemento existe, quando pode ser alterado e como desfazer a alteração.

No React, descrevemos a consequência dos dados:

```
None
function TaskTitle({ title, highlighted }) {
  return (
    <h1 className={highlighted ? "highlight" : ""}>
      {title}
    </h1>
  );
}
```

Quando `title` ou `highlighted` muda, o componente é reavaliado. O React cria uma nova representação de elementos, compara-a com a anterior e atualiza o DOM real conforme a diferença.

4.6 Virtual DOM, reconciliação e renderização

O termo **Virtual DOM** descreve uma representação JavaScript da interface. Ela não é o DOM real do navegador. O fluxo simplificado é:

1. O estado ou as propriedades mudam.
2. O React executa novamente os componentes afetados.
3. Os componentes retornam uma nova árvore de elementos React.
4. O React compara a árvore nova com a anterior. Esse processo é chamado de reconciliação.
5. O React confirma no DOM real apenas as alterações necessárias.

Exemplo: se uma lista possui 100 tarefas e apenas o título de uma muda, o objetivo do React é atualizar o texto correspondente, não reconstruir manualmente todos os nós no navegador.

Isso não significa que o Virtual DOM seja sempre mais rápido em qualquer situação. O benefício principal é oferecer um modelo declarativo e previsível para coordenar atualizações. A performance também depende de chaves estáveis, quantidade de componentes, tamanho das listas e trabalho realizado durante a renderização.

As **keys** ajudam o React a identificar itens de uma coleção:

None

```
{tasks.map((task) => (  
  <li key={task.id}>{task.title}</li>  
))}
```

Use um identificador estável da tarefa. Evite usar o índice do array como `key` quando a lista pode ser reordenada, inserida ou removida, pois isso pode fazer o React associar o estado visual ao item errado.

Exercício 2 - DOM imperativo e React declarativo

Compare as duas abordagens para uma tela que deve mostrar `0` ou `1` tarefa concluída:

1. Descreva quais elementos o código imperativo precisaria localizar e modificar.
2. Descreva quais dados o componente React receberia.
3. Explique qual abordagem tende a ficar mais fácil de manter quando a tela crescer.
4. Abra o DevTools do navegador, inspecione o `<main>` e identifique o DOM real gerado pelo React.

Verificação: sua resposta deve diferenciar a árvore React criada pelo código e o DOM real que o navegador inspeciona.

Exercício 1 - Identificando responsabilidades

Classifique cada item como **frontend**, **backend** ou **banco de dados**:

1. Exibir o título de uma tarefa.
2. Validar se uma tarefa possui título antes de salvá-la.
3. Armazenar a data de criação de uma tarefa.
4. Alterar a cor de um botão ao passar o mouse.
5. Verificar se o usuário tem autorização para excluir uma tarefa.

Verificação: frontend: 1 e 4; backend: 2 e 5; banco de dados: 3. Algumas responsabilidades podem existir em mais de uma camada.

5. Preparando o ambiente

5.1 VS Code

O Visual Studio Code será o editor usado para escrever o projeto. Extensões úteis incluem ESLint, Prettier e Error Lens. Elas ajudam a encontrar problemas e formatar o código, mas não substituem a leitura das mensagens de erro.

5.2 Node.js e npm

O Node.js permite executar JavaScript fora do navegador e fornece ferramentas importantes para o desenvolvimento React. O npm instala bibliotecas, executa scripts e registra dependências no `package.json`.

Abra o terminal e execute:

```
None
node --version
npm --version
```

Você deverá ver os números das versões instaladas.

5.3 Git

Git é um sistema de controle de versões. Ele registra alterações e permite recuperar versões anteriores.

```
None
git --version
git config --global user.name "Seu Nome"
git config --global user.email "seu.email@example.com"
```

Use um endereço de e-mail apropriado para repositórios públicos.

Exercício 2 - Checklist do ambiente

Crie `checklist-ambiente.md`:

None

Checklist do ambiente

- [] VS Code instalado
- [] Node.js funcionando
- [] npm funcionando
- [] Git funcionando
- [] Navegador atualizado
- [] Terminal integrado funcionando

Marque cada item somente depois de executar o comando ou testar a ferramenta.

6. Criando o projeto com Vite

Vite cria a estrutura inicial do projeto e oferece um servidor local rápido. Na pasta onde você guarda seus projetos, execute:

None

```
npm create vite@latest todo-pro
cd todo-pro
npm install
npm run dev
```

Quando o comando fizer perguntas, escolha `React` e `JavaScript`. O terminal exibirá um endereço parecido com `http://localhost:5173/`. Abra esse endereço no navegador. Para encerrar o servidor, pressione `Ctrl + C` no terminal.

`localhost` representa o próprio computador. A porta `5173` identifica o processo do servidor de desenvolvimento, mas pode mudar se já estiver ocupada.

Exercício 3 - Primeiro projeto

1. Crie o projeto `todo-pro`.
2. Execute `npm install`.
3. Execute `npm run dev`.
4. Abra o endereço no navegador.
5. Altere um texto e observe a atualização.
6. Encerre o servidor com `Ctrl + C`.

Verificação: você concluiu quando consegue iniciar e encerrar o projeto sem depender de um arquivo externo.

7. Conhecendo a estrutura do projeto

Item	Função
<code>package.json</code>	Nome, scripts e dependências.
<code>package-lock.json</code>	Versões exatas instaladas pelo npm.
<code>node_modules/</code>	Dependências instaladas; não deve ser versionada.
<code>public/</code>	Arquivos públicos.
<code>src/</code>	Código principal da aplicação.
<code>src/main.jsx</code>	Ponto de entrada que inicializa o React.
<code>src/App.jsx</code>	Componente principal inicial.
<code>src/index.css</code>	Estilos globais.
<code>.gitignore</code>	Arquivos ignorados pelo Git.

A relação inicial é:

None

```
index.html -> main.jsx -> App.jsx -> navegador
```

No `package.json`, o script `dev` normalmente aponta para o Vite:

None

```
"scripts": {  
  "dev": "vite",  
  "build": "vite build",  
  "preview": "vite preview"  
}
```

Exercício 4 - Exploração orientada

Responda em `estrutura-projeto.md`:

1. Qual arquivo contém os scripts do npm?

2. Por que `node_modules` não deve ser enviado ao Git?
3. Onde ficará o código principal?
4. Qual é o papel do `main.jsx`?
5. Qual comando cria uma versão de produção?

Respostas esperadas: `package.json`; porque pode ser recriado com `npm install`; `src`; inicializar/renderizar a aplicação; `npm run build`.

8. Escrevendo JSX

JSX permite escrever uma sintaxe parecida com HTML dentro do JavaScript. Ele é transformado pelo processo de build em código que o React entende.

None

```
function Welcome() {  
  const name = "Equipe To-Do Pro";  
  
  return (  
    <section>  
      <h1>Bem-vindo, {name}</h1>  
      <p>Organize suas tarefas.</p>  
    </section>  
  );  
}
```

Observe que o componente é uma função, seu nome começa com letra maiúscula, JavaScript fica entre chaves e o retorno possui um elemento raiz.

Substitua o conteúdo de `src/App.jsx` por:

None

```
function App() {  
  return (  
    <main>  
      <header>  
        <h1>To-Do Pro</h1>  
        <p>Organize suas tarefas em um só  
lugar.</p>  
      </header>  
  
      <section>  
        <h2>Minhas tarefas</h2>  

```

```

        <p>Nenhuma tarefa cadastrada ainda.</p>
        <button type="button">Adicionar
tarefa</button>
      </section>
    </main>
  );
}

export default App;

```

Salve e observe o navegador. Se houver erro, confira parênteses, chaves, tags, aspas e a existência de um único elemento raiz.

Exercício 6 - Personalizando JSX

Altere a tela para incluir:

1. seu nome ou nome da equipe;
2. uma frase que explique o problema resolvido;
3. o texto `0 tarefas pendentes`;
4. três tarefas de exemplo;
5. um botão chamado `Nova tarefa`.

Use apenas JSX e dados fixos. O botão ainda não precisa funcionar.

Verificação: a tela deve abrir sem erros e apresentar os cinco itens solicitados.

9. Estado local com `useState`

Uma interface real precisa reagir a acontecimentos: o usuário digita, marca uma tarefa, remove um item ou troca um filtro. Essas informações que podem mudar durante a execução são chamadas de **estado**.

No React, o estado local de um componente pode ser criado com o hook `useState`:

```

None
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

```

```

    return (
      <button type="button" onClick={() => setCount(count +
1)}}>
        Cliques: {count}
      </button>
    );
  }

```

A linha abaixo possui três partes:

None

```
const [count, setCount] = useState(0);
```

- `count` é o valor atual do estado;
- `setCount` é a função fornecida pelo React para solicitar uma atualização;
- `0` é o valor inicial;
- a sintaxe entre colchetes usa desestruturação de arrays do JavaScript.

9.1 O que acontece quando o estado muda?

Quando `setCount` é chamada, o React agenda uma nova renderização do componente. Em termos simplificados:

1. o usuário clica no botão;
2. o evento chama `setCount`;
3. o React armazena o novo estado;
4. o componente `Counter` é executado novamente;
5. o JSX agora contém o novo valor de `count`;
6. o React reconcilia a nova árvore com a anterior;
7. o texto do botão no DOM real é atualizado.

O componente ser executado novamente não significa que o DOM inteiro seja apagado e recriado. A função produz uma nova descrição da interface, e o React aplica a menor alteração necessária ao DOM real.

9.2 Estado não é uma variável comum

Uma variável local comum perde seu valor quando a função é executada novamente:

None

```
function WrongCounter() {
  let count = 0;

```

```

    function increment() {
        count += 1;
        console.log(count);
    }

    return <button onClick={increment}>Cliques:
{count}</button>;
}

```

Esse código altera uma variável, mas não solicita uma nova renderização. O console pode mostrar valores, porém o texto na tela não acompanha corretamente a mudança. O `useState` mantém o valor entre renderizações e fornece uma função que comunica ao React que a interface precisa ser reavaliada.

9.3 Comparação com Angular

No Angular, você pode alterar uma propriedade da classe e o mecanismo do framework atualiza o template. No React, a alteração de uma variável comum não é suficiente: para estado local, use o setter retornado pelo hook.

None

Angular: propriedade da classe -> detecção de mudanças/signals -> template
 React: setter do hook -> nova renderização -> reconciliação -> DOM

Não pense no setter como uma atribuição direta. `setCount(3)` é uma solicitação de atualização gerenciada pelo React. O valor pode ser aplicado em lote junto com outras atualizações.

9.4 Atualização baseada no valor anterior

Quando o novo valor depende do valor anterior, prefira a forma funcional:

None

```
setCount((currentCount) => currentCount + 1);
```

Essa forma é mais segura quando há várias atualizações no mesmo evento:

None

```
function Counter() {
```

```

const [count, setCount] = useState(0);

function incrementThreeTimes() {
  setCount((current) => current + 1);
  setCount((current) => current + 1);
  setCount((current) => current + 1);
}

return (
  <>
    <p>Valor: {count}</p>
    <button type="button"
onClick={incrementThreeTimes}>
      Somar 3
    </button>
  </>
);
}

```

O React pode agrupar atualizações para reduzir renderizações. Cada função recebe o resultado mais recente disponível dentro da sequência. A forma `setCount(count + 1)` usa o valor capturado naquela renderização e pode não produzir o resultado esperado quando repetida.

9.5 Imutabilidade de objetos e arrays

O estado deve ser tratado como imutável. Não altere diretamente o array ou objeto guardado no estado:

```

None
// Evite
tasks.push(newTask);
setTasks(tasks);

```

Nesse exemplo, o array continua sendo a mesma referência. Isso dificulta que o React e outras ferramentas percebam a mudança corretamente.

Crie uma nova referência:

```

None
setTasks((currentTasks) => [...currentTasks, newTask]);

```

Para concluir uma tarefa:

None

```
setTasks((currentTasks) =>
  currentTasks.map((task) =>
    task.id === taskId
      ? { ...task, completed: !task.completed }
      : task
  )
);
```

Para remover:

None

```
setTasks((currentTasks) =>
  currentTasks.filter((task) => task.id !== taskId)
);
```

O operador spread (...) copia propriedades ou itens, enquanto `map` e `filter` devolvem novos arrays.

9.6 Estado derivado

Nem todo valor exibido precisa ser armazenado em outro estado. Se ele pode ser calculado a partir de `tasks`, derive-o durante a renderização:

None

```
const completedCount = tasks.filter((task) =>
  task.completed).length;
const pendingCount = tasks.length - completedCount;
```

Evite manter simultaneamente `tasks` e `completedCount` em estados independentes. Dois valores que representam a mesma informação podem ficar inconsistentes. Guarde a fonte original e calcule os indicadores.

9.7 Regras importantes dos hooks

- Chame hooks no nível superior do componente, não dentro de `if`, `for` ou funções aninhadas.
- Chame hooks apenas dentro de componentes React ou de hooks customizados.
- Não use `useState` para esconder um problema de modelagem de dados.
- Escolha o estado mínimo necessário e derive o restante.
- Passe dados para componentes filhos por props; passe funções quando o filho precisar solicitar uma alteração ao pai.

Exemplo: contador de tarefas

None

```
import { useState } from "react";

function TaskSummary() {
  const [tasks, setTasks] = useState([
    { id: 1, title: "Estudar React", completed: false },
    { id: 2, title: "Criar projeto Vite", completed: true },
  ],
  );

  const completedCount = tasks.filter((task) =>
task.completed).length;

  function toggleTask(taskId) {
    setTasks((currentTasks) =>
      currentTasks.map((task) =>
        task.id === taskId
          ? { ...task, completed:
!task.completed }
          : task
      )
    );
  }

  return (
    <section>
      <p>Concluídas: {completedCount}</p>
      <ul>
        {tasks.map((task) => (
          <li key={task.id}>
            <span>{task.title}</span>
            <button type="button"
onClick={() => toggleTask(task.id)}>
              {task.completed ? "Reabrir"
: "Concluir"}
            </button>
          </li>
        ))}
      </ul>
    </section>
  );
}
```



```

        </ul>
      </section>
    );
  }

```

Exercício 7 - Contador e estado de tarefas

Crie um componente `TaskSummary` que:

1. possua um estado com pelo menos três tarefas;
2. mostre o total de tarefas;
3. mostre o total de tarefas concluídas;
4. permita alternar o status de uma tarefa;
5. use `key` com o identificador da tarefa;
6. não use `push`, `splice` ou atribuição direta ao array do estado.

Depois responda:

- por que `setTasks((currentTasks) => ...)` foi usado?
- por que `completedCount` não precisa ser outro `useState`?
- o que muda no DOM real quando você conclui uma tarefa?

Verificação: o contador deve atualizar sem recarregar a página e o console não deve apresentar avisos de `key`.

10. Versionando o trabalho com Git

Na pasta do projeto, execute:

```

None
git init
git status
git add .
git commit -m "feat: cria base inicial do To-Do Pro"
git log --oneline -1

```

Um commit é um registro de um conjunto coerente de alterações. Mensagens úteis indicam o que foi feito, por exemplo `feat: cria tela inicial de tarefas` ou `docs: adiciona instrucoes de execucao`.

Exercício 8 - Primeiro registro

1. Confira `git status`.
2. Confirme que `node_modules` não está sendo listado.
3. Crie o commit da primeira versão.
4. Consulte o histórico com `git log --oneline -1`.

Verificação: seu histórico deve conter um commit identificável da primeira entrega.

11. Documentando o projeto

Crie ou edite `README.md` na raiz:

```
None
# To-Do Pro

Aplicação React para organização de tarefas.

## Pré-requisitos
- Node.js LTS
- npm

## Instalação
```bash
npm install
```

## Execução
```bash
npm run dev
```
```

Um README útil responde: o que é o projeto, o que é necessário e como executá-lo.

Exercício 9 - Teste de reprodução

Peça a um colega para abrir o README e executar o projeto. Depois registre pelo menos uma melhoria feita a partir do teste.

12. Desafio final - Primeira entrega do To-Do Pro

Contexto

A empresa solicitou uma demonstração inicial. A aplicação ainda não precisa salvar dados, mas deve comunicar sua finalidade e mostrar como a futura lista de tarefas funcionará.

Tarefa

Entregue um projeto React com Vite que contenha:

- cabeçalho com `To-Do Pro`;
- descrição curta do sistema;
- seção `Minhas tarefas`;
- pelo menos três tarefas fixas;
- diferença visual entre tarefa pendente e concluída;
- botão `Nova tarefa`;
- layout legível em desktop e janela estreita;
- README de instalação e execução;
- pelo menos dois commits coerentes.

Requisitos técnicos

- usar React e JSX;
- iniciar o projeto com Vite;
- manter o código em `src`;
- utilizar `main`, `header`, `section` e `button`;
- não utilizar backend ou banco nesta etapa;
- não inserir senhas ou tokens no repositório;
- executar com `npm run dev` sem erros no console.

Roteiro de execução

1. Organize a tela no `App.jsx`.
2. Se desejar, crie um componente `TaskPreview`.
3. Adicione estilos em `src/index.css`.
4. Teste a aplicação no navegador.
5. Revise o console e o terminal.
6. Atualize o README.
7. Faça commits descritivos.

Checklist de entrega

O projeto executa com `npm install` e `npm run dev`.
O nome To-Do Pro aparece na tela.
A finalidade está descrita.
Existem pelo menos três tarefas.

Há diferença visual entre pendente e concluída.
A interface usa estrutura semântica.
Não há erro no console.
O README explica instalação e execução.
O repositório possui pelo menos dois commits.
Consigo explicar a estrutura do projeto.

13. Autoavaliação

| Afirmação | Ainda não | Com ajuda | Sozinho |
|---|-----------|-----------|---------|
| Consigo verificar Node.js, npm e Git. | | | |
| Consigo criar um projeto React com Vite. | | | |
| Consigo iniciar e encerrar o servidor local. | | | |
| Consigo explicar <code>src</code> , <code>package.json</code> e <code>node_modules</code> . | | | |
| Consigo criar um componente com JSX. | | | |
| Consigo registrar alterações com Git. | | | |
| Consigo escrever um README de execução. | | | |
| Consigo investigar um erro simples. | | | |

Responda também:

1. Qual foi o erro mais difícil da semana?
2. Como você investigou esse erro?
3. Qual parte da entrega ainda pode melhorar?
4. O que você espera aprender na próxima semana?

14. Problemas comuns

node: command not found: Node.js não está instalado ou não foi adicionado ao PATH.
Instale a versão LTS, reinicie o terminal e tente novamente.

Falha ao criar o projeto: confira a internet, a versão do npm e o diretório atual. Não crie um projeto dentro de outro projeto.

Porta em uso: use o endereço alternativo exibido pelo Vite ou encerre o processo anterior com `Ctrl + C`.

Página não atualiza: salve o arquivo, confirme que o servidor está ativo e verifique se editou o projeto correto.

Erro de JSX: confira tags fechadas, elemento raiz único, chaves, parênteses, aspas e nomes de atributos.

Arquivos demais no Git: verifique o `.gitignore`. A pasta `node_modules` não deve ser versionada.

15. Glossário rápido

- **API:** interface usada para comunicação entre sistemas.
- **Componente:** unidade reutilizável de interface React.
- **Frontend:** camada visual e interativa.
- **JSX:** sintaxe que combina JavaScript e marcação semelhante a HTML.
- **npm:** gerenciador de pacotes e scripts do Node.js.
- **React:** biblioteca JavaScript para interfaces.
- **SPA:** aplicação que atualiza a interface sem recarregar tudo.
- **Vite:** ferramenta de criação e desenvolvimento frontend.

16. Referências

- React. **Learn React.** <https://react.dev/learn>
- Vite. **Getting Started.** <https://vite.dev/guide/>
- Node.js. **Documentação oficial.** <https://nodejs.org/docs/latest/api/>
- Git. **Documentação oficial.** <https://git-scm.com/doc>
- MDN Web Docs. **JavaScript.** <https://developer.mozilla.org/pt-BR/docs/Web/JavaScript>